

Pila / Stack

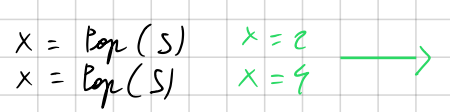
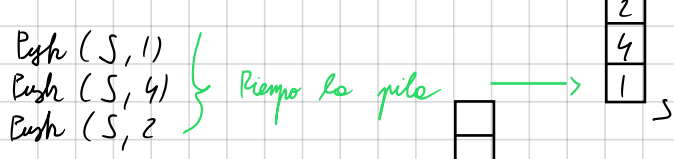
Cos'è? : È un tipo di dato astratto per insieme dinamico in cui inserimenti e cancellazioni sono regolati dalla politica **LIFO** (Last In First Out)

Operazioni:

- 1) Inserimento / Push $\rightarrow \text{Push}(S, x) \rightarrow S$
- 2) cancellazione / Pop $\rightarrow \text{Pop}(S) \rightarrow x$
- 3) query / accesso $\rightarrow \text{Top}(S) \rightarrow x$ restituisce il prossimo elemento per rimuoverlo
- 4) query : $\text{Stack Empty}(S) \rightarrow \text{T/F}$

Esempio

$S = \text{pila vuota}$



$\text{Push}(S, 5)$

$x = \text{Pop}(S)$

$x = \text{Pop}(S)$

$x = \text{Pop}(S)$ *errore underflow, la pila è vuota*

Code / Queue

Logica: Stessa cosa della pila ma ha ordine FIFO (First In First Out)

Operazioni

1) Enqueue (Q, x) $\rightarrow$ Q

2) Dequeue (Q) $\rightarrow$ x

3) Head (Q) $\rightarrow$ x

4) Queue_Empty (Q) $\rightarrow$ T/F

Esempio

Q = nuova coda vuota

Enqueue (Q, 1)

Enqueue (Q, 4)

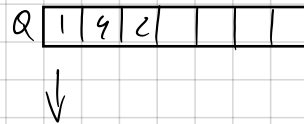
Enqueue (Q, 2)

x = Dequeue (Q)

x = Dequeue (Q)

Enqueue (Q, 5)

x = Dequeue (Q) x 3 volte



procedure **Stack Search** (S, x)

$S_1 =$ nuovo pile vuoto

While not Stack Empty (S) and $x \neq \text{Top}(S)$ do

$y = \text{Pop}(S)$

Push (S_1, y)

trovato = not Stack Empty (S)

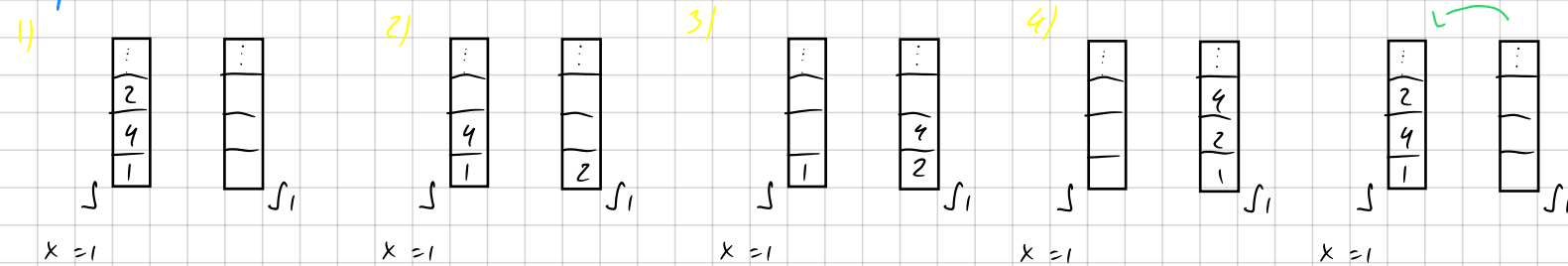
$x == \text{Top}(S)$?

While not Stack Empty (S_1) do

Push ($S, \text{Pop}(S_1)$)

return trovato

Esempio



trovato = true

Tempi

$T_{\text{in}}(n) = \Theta(1)$ elemento ricercato in cima

$T_{\text{pg}}(n) = \Theta(n)$ elemento ricercato in fondo o non presente

procedura *QueueSearch* (Q, x)

Q

2	4	1	3	...
---	---	---	---	-----

 $x=1$
 $y=1$

per trovare le stesse code di portenze dovrò smontare le code di portenza

Q

--	--	--	--	--

Q_1

2	4	1	3
---	---	---	---

Q

2	4	1	3
---	---	---	---

procedura *QueueSearch* (Q, x)

Q_1 = nuova code vuota

trovato = FALSE

while not *QueueEmpty* (Q) do

$y = \text{Dequeue}(Q)$

 if $x == y$ then trovato = true

Enqueue (Q_1, y)

while not *QueueEmpty* (Q_1) do

Enqueue ($Q, \text{Dequeue}(Q_1)$)

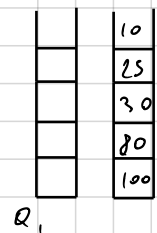
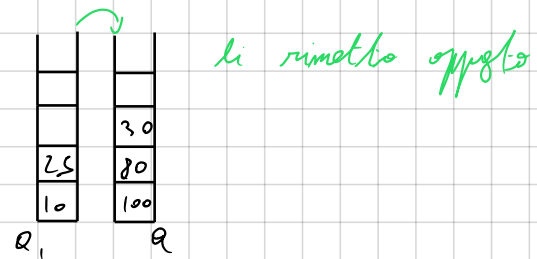
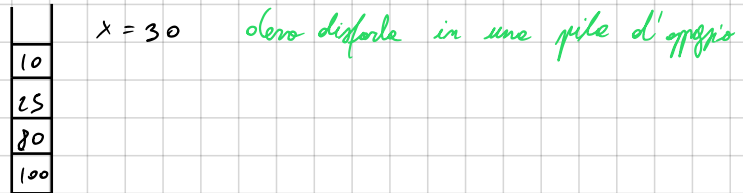
return trovato

// le operazioni richieste e le loro implementazione non cruciali al calcolo dei tempi

$T(n) = \Theta(n)$

↑
dipende dall'implementazione della queue

Inserire un elemento mantenendo l'ordine della pila



procedura **Stack Sort Insert** (S, x)

$S_1 =$ pila vuota

While not Stack Empty (S) and $Top(S) < x$ do

 Push (S_1 , Pop (S))

Push (S, x)

While not Stack Empty (S_1) do

 Push (S , Pop (S_1))

$$T_{insl}(n) = \Theta(1)$$

$$x < Top(S)$$

$$T_{ps}(n) = \Theta(n)$$

x è più grande di ogni elemento della pila

Stack Sort Insert (S, x, y)

if $y < x$ then scramble x & y

S_1 pile vuoto

while not StackEmpty(S) and $\text{top}(S) < x$ do

Push(S_1 , Pop(S))

while not StackEmpty(S) and $\text{top}(S) < y$ do

Push(S_1 , Pop(S))

Push(S , y)

while not StackEmpty(S_1) do

Push(S , Pop(S_1))

Coda implementata con array circolare

Cos'è?: Una coda ad array circolare è un'implementazione della coda tramite array, in cui gli indici head e tail si muovono in modo circolare all'interno dell'array.

Come funziona?: Una coda ad array circolare sfrutta un array fisso per gestire in modo efficiente inserimenti e rimozioni, riutilizzando gli spazi liberi all'inizio dell'array senza spostare elementi, per farlo, mantiene 2 indici:

- **head**: indica la testa della coda, da cui si estrae
- **tail**: indica la posizione successiva in cui inserire

Quando tail o head raggiungono la fine dell'array, si riportano a zero grazie all'aritmetica modulare.

- $head = (head + 1) \% Q.length$
- $tail = (tail + 1) \% Q.length$

Esempio

1) $Q = [\downarrow, \uparrow, , , ,]$
↓ head = 0
↑ tail = 0

2) $Q = [\downarrow 10, \uparrow, , ,]$
enqueue(10)
↓ head = 0
↑ tail = 1

3) $Q = [\downarrow 10, 20, \uparrow, ,]$
enqueue(20)
↓ head = 0
↑ tail = 2

4) $Q = [, \downarrow 20, \uparrow, ,]$
enqueue(20)
↓ head = 1
↑ tail = 2

5) $Q = [\uparrow, , \downarrow 20, 30, 40, 50]$
enqueue(30)
enqueue(40)
enqueue(50)
↓ head = 1
↑ tail = (tail + 1) \% Q.length = (4 + 1) \% 5 = 0

6) $Q = [60, \downarrow 20, \uparrow, 30, 40, 50]$
enqueue(60)
↓ head = 1
↑ tail = 1

Lo coda è piena, rischio overflow!

Implementazione

class CircularQueue

```
int[] Q  
int head  
int tail  
int size // numero corrente di elementi  
int capacity // capacità dell'array
```

```
CircularQueue (int capacity)
```

```
    this.capacity = capacity  
    Q = new int[capacity]
```

```
    head = 0  
    tail = 0  
    size = 0
```

Inserimento - enqueue (x)

```
void enqueue (int x)
```

```
    // Controllo overflow
```

```
    if isFull() throw error "overflow"
```

```
    Q[tail] = x
```

```
    // inserimento in posizione tail
```

```
    tail = (tail + 1) % capacity
```

```
    // tail avanza circolarmente
```

```
    size++
```

```
    // aggiorna il conteggio
```

Tempi di calcolo: $\Theta(1)$

Sono tutte operazioni a tempo costante

Esulsione - dequeue ()

```
int dequeue ()
```

```
    // Controllo underflow
```

```
    if isEmpty() throw error "underflow"
```

```
    value = Q[head]
```

```
    // Leggi elemento in head
```

```
    head = (head + 1) % capacity
```

```
    // head avanza
```

```
    size--
```

```
    // aggiorna il conteggio
```

```
    return value // Output
```

Tempi di calcolo: $\Theta(1)$

Sono tutte operazioni a tempo costante

Metodi: isEmpty() e isFull()

```
boolean isEmpty()  
return size == 0
```

```
boolean isFull()  
return size == capacity
```

Metodo peek()

```
int peek()  
// Controllo underflow  
if isEmpty() throw error "underflow"  
return Q[head] // output
```

Tempi di calcolo: $\Theta(1)$

Sono tutte operazioni a tempo costante

Tempi di calcolo: $\Theta(1)$

Sono tutte operazioni a tempo costante